

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

CRUD e transazioni con MySQL

Leggere, analizzare, collegare e modificare i dati, controllando le operazioni con `COMMIT` e `ROLLBACK`.

CAPITOLO 1

Le quattro operazioni CRUD

NOTA

CRUD riassume le quattro operazioni fondamentali sui dati: Create, Read, Update e Delete.

- **Create:** creare un nuovo record con `INSERT INTO`.
- **Read:** leggere uno o più record con `SELECT`.
- **Update:** modificare record esistenti con `UPDATE`.
- **Delete:** eliminare record con `DELETE`.

Gli esempi usano una tabella `products` già esistente con queste colonne:

products			
+-----+-----+-----+			
Colonna	Tipo	Significato	
+-----+-----+-----+			
id	INT	Identificativo del prodotto	
name	VARCHAR(255)	Nome del prodotto	
price	DECIMAL(8,2)	Prezzo	
stock_quantity	INT	Quantità disponibile	
is_deleted	INT(1)	0 = attivo, 1 = eliminato	
created_at	DATETIME	Data di creazione	
updated_at	DATETIME	Data dell'ultima modifica	
+-----+-----+-----+			

Ogni istruzione SQL termina con un punto e virgola. Le parole chiave sono scritte in maiuscolo per riconoscerle più facilmente.

CAPITOLO 2

Leggere i dati con SELECT

NOTA

SELECT legge i dati senza modificarli. È l'istruzione da usare anche prima di un aggiornamento o di una cancellazione.

Dopo **SELECT** indichiamo le colonne da leggere; dopo **FROM** specifichiamo la tabella. Il carattere *****, chiamato asterisco, seleziona **tutte le colonne**:

```
SELECT *  
FROM `products`;
```

SELECT * è comodo per esplorare rapidamente una tabella. Nelle query dell'applicazione è però preferibile elencare soltanto le colonne necessarie: il risultato è più chiaro e il database trasferisce meno dati.

Per leggere soltanto identificativo, nome e prezzo scriviamo:

```
SELECT  
    `id`,  
    `name`,  
    `price`  
FROM `products`;
```

Possiamo usare **WHERE** per filtrare i risultati:

```
SELECT  
    `id`,  
    `name`,  
    `price`  
FROM `products`  
WHERE `is_deleted` = 0  
    AND `price` < 50.00;
```

- **WHERE** conserva soltanto i record che rispettano una condizione.
- **AND** richiede che entrambe le condizioni siano vere.

CAPITOLO 3

Inserire dati con INSERT INTO

NOTA

INSERT INTO crea un nuovo record. L'ordine dei valori deve corrispondere esattamente all'ordine delle colonne.

```
INSERT INTO `products` (  
    `name`,  
    `price`,  
    `stock_quantity`  
) VALUES (  
    'Tastiera meccanica',  
    79.90,  
    12  
);
```

Non inseriamo **id** perché è generato automaticamente. Omettiamo anche le colonne di log e **is_deleted**, che ricevono i propri valori predefiniti.

Le stringhe usano gli apici singoli, mentre i numeri non li richiedono. Per inserire più record separiamo i gruppi di valori con una virgola:

```
INSERT INTO `products` (  
    `name`,  
    `price`,  
    `stock_quantity`  
) VALUES  
    ('Mouse wireless', 29.90, 20),  
    ('Webcam HD', 54.50, 8);
```

CAPITOLO 4

Modificare dati con UPDATE

NOTA

UPDATE modifica record già esistenti. Senza **WHERE**, modifica tutti i record della tabella.

Prima controlliamo con **SELECT** quali record corrispondono alla condizione:

```
SELECT
    `id`,
    `name`,
    `price`
FROM `products`
WHERE `id` = 1;
```

Se il record è quello corretto, possiamo modificarlo usando lo stesso **WHERE** :

```
UPDATE `products`
SET
    `price` = 69.90,
    `stock_quantity` = 15
WHERE `id` = 1;
```

SET contiene le nuove assegnazioni. Possiamo modificare una o più colonne, separandole con una virgola.

MySQL aggiorna automaticamente **updated_at** se la colonna usa **ON UPDATE CURRENT_TIMESTAMP**.

CAPITOLO 5

Eliminare dati con DELETE

NOTA

DELETE elimina record. Senza **WHERE**, elimina tutti i record della tabella.

Come per **UPDATE**, eseguiamo prima un **SELECT** con la stessa condizione:

```
SELECT
    `id`,
    `name`
FROM `products`
WHERE `id` = 8;
```

Dopo aver verificato il record, possiamo eliminarlo definitivamente:

```
DELETE FROM `products`
WHERE `id` = 8;
```

Prima di un **DELETE** bisogna controllare anche le chiavi esterne e le relative azioni **ON DELETE**: una cancellazione potrebbe essere bloccata oppure propagarsi ad altre tabelle.

Quando è utile conservare la cronologia, possiamo applicare una **cancellazione logica**: il record resta nel database, ma viene marcato come eliminato.

```
UPDATE `products`
SET
    `is_deleted` = 1
WHERE `id` = 8;
```

Le normali letture dovranno quindi filtrare i record con **WHERE `is\_deleted` = 0**.

CAPITOLO 6

I tipi di JOIN

NOTA

Una **JOIN** collega i record di due tabelle attraverso colonne correlate, solitamente una chiave primaria e una chiave esterna.

INNER JOIN

INNER JOIN restituisce soltanto i record che trovano una corrispondenza in entrambe le tabelle:

```
SELECT
    `products`.`id`,
    `products`.`name`,
    `categories`.`name`
FROM `products`
INNER JOIN `categories`
    ON `categories`.`id` = `products`.`id_category`;
```

La parola **INNER** può essere omessa: scrivere soltanto **JOIN** produce lo stesso risultato.

LEFT JOIN

LEFT JOIN mantiene tutti i record della tabella a sinistra, cioè **products**. Se un prodotto non ha una categoria collegata, le colonne di **categories** contengono **NULL**.

```
SELECT
    `products`.`id`,
    `products`.`name`,
    `categories`.`name`
FROM `products`
LEFT JOIN `categories`
    ON `categories`.`id` = `products`.`id_category`;
```

RIGHT JOIN

RIGHT JOIN mantiene tutti i record della tabella a destra, cioè **categories**. Una categoria priva di prodotti compare comunque; le colonne di **products** contengono **NULL**.

```
SELECT
    `products`.`id`,
    `products`.`name`,
    `categories`.`name`
FROM `products`
RIGHT JOIN `categories`
    ON `categories`.`id` = `products`.`id_category`;
```

Spesso una **RIGHT JOIN** viene riscritta come **LEFT JOIN** invertendo l'ordine delle tabelle, perché risulta più immediata da leggere.

NATURAL JOIN

NATURAL JOIN crea automaticamente il collegamento usando tutte le colonne che hanno lo stesso nome nelle due tabelle. Per questo motivo non richiede la condizione **ON**:

```
SELECT *
FROM `products`
NATURAL JOIN `categories`;
```

Nel nostro schema le due tabelle hanno diverse colonne con lo stesso nome. **NATURAL JOIN** le userebbe tutte per creare il collegamento. È quindi più sicuro specificare la relazione con **ON** quando vogliamo controllare con precisione le colonne confrontate.

CAPITOLO 7

Calcolare valori con le funzioni SQL

NOTA

Le **funzioni aggregate** ricevono i valori di più record e producono un risultato riassuntivo.

- **COUNT** conta i record o i valori presenti.
- **SUM** somma i valori di una colonna numerica.
- **AVG** calcola la media dei valori.
- **MIN** trova il valore più piccolo.
- **MAX** trova il valore più grande.

Possiamo usare più funzioni nella stessa query:

```
SELECT
    COUNT(`id`) AS `product_count`,
    SUM(`stock_quantity`) AS `total_stock`,
    AVG(`price`) AS `average_price`,
    MIN(`price`) AS `lowest_price`,
    MAX(`price`) AS `highest_price`
FROM `products`
WHERE `is_deleted` = 0;
```

Il **WHERE** filtra prima i record; le funzioni lavorano soltanto sui record rimasti. **AS** assegna un alias leggibile a ogni colonna del risultato.

CAPITOLO 8

Filtrare i gruppi con HAVING

NOTA

HAVING filtra i risultati prodotti da **GROUP BY** e dalle funzioni aggregate.

WHERE filtra i singoli record prima del raggruppamento; **HAVING** filtra i gruppi dopo che MySQL ha calcolato funzioni come **COUNT** e **AVG**.

```
SELECT
    `id_category`,
    COUNT(`id`) AS `product_count`,
    AVG(`price`) AS `average_price`
FROM `products`
WHERE `is_deleted` = 0
GROUP BY `id_category`
HAVING
    COUNT(`id`) >= 5
    AND AVG(`price`) > 50.00;
```

La query crea un gruppo per ogni categoria e conserva soltanto le categorie con almeno cinque prodotti attivi e un prezzo medio superiore a 50 euro.

CAPITOLO 9

Ordinare i risultati con ORDER BY

NOTA

ORDER BY ordina i record restituiti da una query in base a una o più colonne.

- **ASC** ordina in modo crescente ed è la direzione predefinita.
- **DESC** ordina in modo decrescente.

Questa query mostra prima i prodotti più costosi:

```
SELECT
  `id`,
  `name`,
  `price`
FROM `products`
WHERE `is_deleted` = 0
ORDER BY `price` DESC;
```

Possiamo indicare più colonne. MySQL usa la seconda quando due record hanno lo stesso valore nella prima:

```
SELECT
  `id`,
  `name`,
  `price`
FROM `products`
WHERE `is_deleted` = 0
ORDER BY
  `price` DESC,
  `name` ASC;
```

CAPITOLO 10

Limitare i risultati con LIMIT

NOTA

LIMIT stabilisce il numero massimo di record restituiti dalla query.

Per ottenere soltanto i primi cinque prodotti:

```
SELECT
    `id`,
    `name`,
    `price`
FROM `products`
ORDER BY `id` ASC
LIMIT 5;
```

OFFSET indica quanti record saltare prima di iniziare a restituire i risultati. È utile per dividere un elenco in pagine:

```
SELECT
    `id`,
    `name`,
    `price`
FROM `products`
ORDER BY `id` ASC
LIMIT 10 OFFSET 20;
```

La query salta i primi venti record e restituisce i dieci successivi, cioè la terza pagina di un elenco da dieci elementi. Con **LIMIT** è importante usare **ORDER BY**, altrimenti l'ordine dei record non è garantito.

CAPITOLO 11

TRANSACTION, COMMIT e ROLLBACK

NOTA

Una **transazione** raggruppa più operazioni: possiamo confermarle tutte insieme oppure annullarle tutte insieme.

Normalmente MySQL usa l'**autocommit**: ogni istruzione completata con successo viene confermata subito.

START TRANSACTION apre invece una transazione esplicita.

- **COMMIT** conferma definitivamente tutte le modifiche della transazione.
- **ROLLBACK** annulla le modifiche non ancora confermate.

In questo esempio cambiamo prezzo e quantità dello stesso prodotto. Dopo aver controllato il risultato, confermiamo entrambe le modifiche:

```
START TRANSACTION;

UPDATE `products`
SET
    `price` = 64.90
WHERE `id` = 1;

UPDATE `products`
SET
    `stock_quantity` = `stock_quantity` + 5
WHERE `id` = 1;

SELECT
    `id`,
    `name`,
    `price`,
    `stock_quantity`
FROM `products`
WHERE `id` = 1;

COMMIT;
```

Se un'istruzione fallisce oppure il risultato del **SELECT** non è corretto, non eseguiamo **COMMIT**: usiamo **ROLLBACK**.

Durante una prova possiamo annullare una modifica volontariamente:

```
START TRANSACTION;

UPDATE `products`
SET
    `price` = `price` * 0.90
WHERE `is_deleted` = 0;

SELECT
    `id`,
    `name`,
    `price`
FROM `products`
WHERE `is_deleted` = 0;

ROLLBACK;
```

Dopo **ROLLBACK**, i prezzi tornano ai valori precedenti. Per ogni transazione scegliamo una sola conclusione: **COMMIT** oppure **ROLLBACK**.

NOTA

Gli esempi richiedono tabelle con motore **InnoDB**. I comandi che cambiano la struttura del database possono causare un commit implicito: qui usiamo le transazioni per **INSERT**, **UPDATE** e **DELETE**.